Mês 9 — Avaliação Completa + Contribuição de Sub-bandas (mai/2027)

Guia de Estudos - PIBITI

Thiago Schiavi

Maio de 2027

Sumário

1 O que o plano exige **2**

2 Avaliação de Todas as Combinações **3**
2.1 Métricas: EER e min-DCF 3
2.2 Loop de Avaliação 4

3 Feature Importance (XGBoost / Random Forest) **6**
3.1 Extraindo as Importâncias 6
3.2 Mapeando *Features* para Sub-bandas 6

4 Permutation Importance **8**
4.1 Implementação 8

5 Atenção às Altas Frequências **10**

6 Curvas ROC **11**

7 Curvas DET **13**

8 Distribuição de Scores **14**

9 EER por Sub-banda (Ablation Study) **16**

10 Figuras Finais **18**

11 Brincadeiras (4 Experimentos) **20**

12 Checklist **22**

13 Desafio Final **23**

Capítulo 1

O que o plano exige

Este mês foca em avaliar exaustivamente todas as combinações que rodamos até agora e entender *por que* algumas funcionam melhor que outras. Não basta obter os resultados; precisamos interpretar a relevância das features (importância de variáveis) e o papel específico de sub-bandas individuais, principalmente nas altas frequências onde os artefatos de síntese costumam residir.

Objetivos do Mês 9:

- Avaliar todas as combinações e analisar o que mais discrimina *bonafide* de *spoof*.
- Calcular o EER e min-DCF in-domain para cada combinação de *corpus* × nível × wavelet × classificador.
- Realizar uma análise de contribuição: identificar as sub-bandas, descritores, níveis e wavelets-mãe mais discriminativos via feature importance e permutation importance.
- Dar especial atenção às altas frequências.
- Gerar figuras finais de qualidade de publicação (DET, ROC, barras de EER).

Marco / Entrega

Marco / Entrega: Avaliação completa e análise de contribuição de sub-bandas/descritores concluídas e documentadas com gráficos e tabelas.

Capítulo 2

Avaliação de Todas as Combinações

Nesta fase, faremos um loop sobre todas as configurações que treinamos. Supondo que tenhamos salvo as previsões (scores e rótulos verdadeiros) em um arquivo, vamos carregar e processar tudo.

2.1 Métricas: EER e min-DCF

Primeiro, definimos funções para calcular o EER (Equal Error Rate) e o min-DCF (Minimum Detection Cost Function).

Listing 2.1: Funções para cálculo de EER e min-DCF

```
import numpy as np
from sklearn.metrics import roc_curve
import pandas as pd
import matplotlib.pyplot as plt

def compute_eer(y_true, y_score):
    """Calcula o Equal Error Rate (EER)"""
    fpr, tpr, thresholds = roc_curve(y_true, y_score, pos_label=1)
    fnr = 1 - tpr
    eer_threshold = thresholds[np.nanargmin(np.absolute((fnr - fpr)
        ))]
    eer = fpr[np.nanargmin(np.absolute((fnr - fpr)))]
    return eer, eer_threshold

def compute_min_dcf(y_true, y_score, p_target=0.01, c_miss=1, c_fa
    =1):
    """Calcula o min-DCF (Minimum Detection Cost Function)"""
    fpr, tpr, thresholds = roc_curve(y_true, y_score, pos_label=1)
    fnr = 1 - tpr
    dcf = c_miss * fnr * p_target + c_fa * fpr * (1 - p_target)
    min_dcf = np.min(dcf)
    return min_dcf
```

Listing 2.2: Criando dados simulados para teste das métricas

```
# Gerando dados simulados para as metricas
np.random.seed(42)
```

```
n_samples = 1000
# 1 = spoof (target positivo para o classificador), 0 = bonafide
y_true_mock = np.random.randint(0, 2, n_samples)
# Scores com algum poder discriminativo
y_score_mock = y_true_mock * np.random.normal(0.7, 0.2, n_samples)
   + \
               (1 - y_true_mock) * np.random.normal(0.3, 0.2,
                  n_samples)

eer_val, eer_th = compute_eer(y_true_mock, y_score_mock)
min_dcf_val = compute_min_dcf(y_true_mock, y_score_mock)

print(f"EER Mock: {eer_val:.4f}")
print(f"Min-DCF Mock: {min_dcf_val:.4f}")
```

2.2 Loop de Avaliação

Suponha que os resultados dos experimentos estejam armazenados em um DataFrame ou arquivo CSV, onde cada linha é uma inferência.

Listing 2.3: Simulando um DataFrame de resultados de todos os experimentos

```
# Simulacao de resultados agregados
experiments = []
wavelets = ['db2', 'db4', 'sym4']
levels = [3, 4, 5]
classifiers = ['RF', 'XGB', 'SVM']

for w in wavelets:
    for l in levels:
        for c in classifiers:
            # Simulando EER e DCF falsos para ilustrar
            eer = np.random.uniform(0.01, 0.15)
            dcf = eer * 1.5 + np.random.uniform(0, 0.05)
            experiments.append({
                'wavelet': w,
                'level': l,
                'classifier': c,
                'eer': eer,
                'min_dcf': dcf
            })

df_results = pd.DataFrame(experiments)
print(df_results.head())
```

Listing 2.4: Encontrando as melhores configurações

```
# Ordenando pelos melhores resultados (menor EER)
best_configs = df_results.sort_values(by='eer').reset_index(drop=
   True)
print("Top 5 Configs:")
```

```
print(best_configs.head(5))
```

Listing 2.5: Exportando tabela LaTeX das melhores configs

```
# Gerar tabela LaTeX formatada
latex_table = best_configs.head(10).to_latex(index=False,
    float_format="%.4f")
print(latex_table)
```

Capítulo 3

Feature Importance (XGBoost / Random Forest)

Classificadores baseados em árvores (*Random Forest*, *XGBoost*) possuem métodos embutidos para calcular a importância das *features*. Vamos extrair essa informação para mapear quais estatísticas de quais sub-bandas são mais cruciais.

3.1 Extraindo as Importâncias

Listing 3.1: Treinando um Random Forest e extraindo Feature Importance

```
from sklearn.ensemble import RandomForestClassifier

# Simulando um vetor de features (e.g., N sub-bandas x 4
    descritores = 32 features)
num_features = 32
X_train_mock = np.random.rand(800, num_features)
y_train_mock = np.random.randint(0, 2, 800)

rf_clf = RandomForestClassifier(n_estimators=50, random_state=42)
rf_clf.fit(X_train_mock, y_train_mock)

importances = rf_clf.feature_importances_
print("Top 5 raw importances:", sorted(importances, reverse=True)
    [:5])
```

3.2 Mapeando *Features* para Sub-bandas

Listing 3.2: Criando nomes descritivos para as *features*

```
# Nomes das estatisticas
stats_names = ['mean', 'var', 'skew', 'kurt']
num_subbands = 8 # Nivel 3 da WPT = 8 sub-bandas
feature_names = []

for sb in range(num_subbands):
```

```
    for st in stats_names:
        feature_names.append(f"sb{sb}_{st}")

print("Exemplo de nomes:", feature_names[:8])
```

Listing 3.3: Mapeando e plotando importância por descritor

```
importances_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
})
importances_df = importances_df.sort_values(by='Importance',
    ascending=False)

# Visualizando o top 10
plt.figure(figsize=(10, 6))
plt.barh(importances_df['Feature'][:10][::-1], importances_df['
    Importance'][:10][::-1])
plt.xlabel('Importance')
plt.title('Top 10 Features (Random Forest)')
plt.tight_layout()
# plt.show()
print("Plot gerado em memoria")
```

Listing 3.4: Agregando importância por Sub-banda

```
# Agrupando por sub-banda (somando a importancia das 4 estatisticas
    )
importances_df['Subband'] = importances_df['Feature'].apply(lambda
    x: x.split('_')[0])
subband_importance = importances_df.groupby('Subband')['Importance'
    ].sum().sort_values(ascending=False)

print("Importancia total por sub-banda:")
print(subband_importance)
```

Capítulo 4

Permutation Importance

A *Feature Importance* nativa das árvores pode ser viesada por *features* com alta cardinalidade. A ***Permutation Importance*** é uma métrica agnóstica a modelos que mede a queda de performance quando embaralhamos os valores de uma coluna no conjunto de teste.

4.1 Implementação

Listing 4.1: Calculando Permutation Importance

```
from sklearn.inspection import permutation_importance
from sklearn.metrics import make_scorer

# Precisamos de dados de teste (validacao)
X_test_mock = np.random.rand(200, num_features)
y_test_mock = np.random.randint(0, 2, 200)

# Scorer customizado usando min_DCF ou acuracia (aqui acuracia por simplicidade)
result = permutation_importance(rf_clf, X_test_mock, y_test_mock,
                                n_repeats=10, random_state=42,
                                   n_jobs=-1)

perm_importances = result.importances_mean
```

Listing 4.2: Formatando os resultados da Permutation Importance

```
perm_df = pd.DataFrame({
    'Feature': feature_names,
    'Perm_Importance': perm_importances,
    'Std': result.importances_std
})

perm_df = perm_df.sort_values(by='Perm_Importance', ascending=False)
print("Top 5 Permutation Importances:")
print(perm_df.head(5))
```

Listing 4.3: Plotando a Permutation Importance

```
plt.figure(figsize=(10, 6))
top_10 = perm_df.head(10).sort_values(by='Perm_Importance',
    ascending=True)

plt.barh(top_10['Feature'], top_10['Perm_Importance'], xerr=top_10[
    'Std'], capsize=5)
plt.xlabel('Queda na Acuracia')
plt.title('Permutation Importance (Top 10)')
plt.tight_layout()
print("Plot de Permutation Importance gerado.")
```

Capítulo 5

Atenção às Altas Frequências

Vocoders tendem a deixar artefatos espúrios e imprecisões de fase nas altas frequências, que são menos perceptíveis ao ouvido humano (menos penalizados pelas funções de perda baseadas no espectro Mel) mas capturáveis pela análise em árvore completa da WPT.

Listing 5.1: Identificando se as sub-bandas importantes são de alta frequência

```
# Supondo Nivel 3: sb0 a sb7. sb0 e' a mais baixa, sb7 a mais alta (dependendo da ordem de Gray/Paley).
# Vamos mapear a frequencia central nominal.
# Se fs = 16kHz, Nyquist = 8kHz. Largura de banda aprox = 1kHz por sub-banda.
bandwidth = 8000 / num_subbands

# Assumindo ordem sequencial de frequencias (natural ordering approximado)
sb_freqs = {f"sb{i}": (i * bandwidth, (i+1) * bandwidth) for i in range(num_subbands)}

for sb, (f_low, f_high) in sb_freqs.items():
    imp = subband_importance.get(sb, 0)
    print(f"{sb} [{f_low:.0f}-{f_high:.0f} Hz]: Imp = {imp:.4f}")
```

Listing 5.2: Visualizando importância vs faixa de frequência

```
freq_labels = [f"{sb_freqs[f'sb{i}'][0]:.0f}-{sb_freqs[f'sb{i}'][1]:.0f}" for i in range(num_subbands)]
imp_values = [subband_importance.get(f'sb{i}', 0) for i in range(num_subbands)]

plt.figure(figsize=(12, 5))
plt.bar(freq_labels, imp_values, color='coral')
plt.xticks(rotation=45, ha='right')
plt.ylabel('Importancia Agregada')
plt.xlabel('Faixa de Frequencia Estimada (Hz)')
plt.title('Importancia vs. Faixa de Frequencia (Nivel 3)')
plt.tight_layout()
```

Capítulo 6

Curvas ROC

A Curva ROC (Receiver Operating Characteristic) mostra a razão de Verdadeiros Positivos (TPR) contra a razão de Falsos Positivos (FPR).

Listing 6.1: Calculando e plotando uma Curva ROC

```
from sklearn.metrics import auc

fpr, tpr, _ = roc_curve(y_true_mock, y_score_mock, pos_label=1)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC
    = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
print("Curva ROC plotada.")
```

Listing 6.2: Comparando múltiplos classificadores na ROC

```
# Simulando 3 scores de diferentes classificadores
scores = {
    'RF': y_score_mock,
    'XGB': y_true_mock * np.random.normal(0.75, 0.2, n_samples) +
       (1 - y_true_mock) * np.random.normal(0.25, 0.2, n_samples),
    'SVM': y_true_mock * np.random.normal(0.65, 0.3, n_samples) +
       (1 - y_true_mock) * np.random.normal(0.35, 0.3, n_samples)
}

plt.figure(figsize=(8,8))
for name, score in scores.items():
    f, t, _ = roc_curve(y_true_mock, score, pos_label=1)
    a = auc(f, t)
    plt.plot(f, t, lw=2, label=f'{name} (AUC = {a:.3f})')

```

```
plt.plot([0, 1], [0, 1], 'k--', lw=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.legend()
plt.title('Comparacao de ROC')
```

Capítulo 7

Curvas DET

A Curva DET (Detection Error Tradeoff) plota FPR contra FNR, frequentemente usando escalas logarítmicas ou transformadas normais (escala de desvios da normal). É a figura padrão-ouro na avaliação de sistemas biométricos (ASVspoof a exige).

Listing 7.1: Usando `DetCurveDisplay` do Scikit-Learn

```
from sklearn.metrics import DetCurveDisplay

plt.figure(figsize=(8,8))
fig, ax = plt.subplots(figsize=(8,8))

# Plotando para os tres classificadores
for name, score in scores.items():
    DetCurveDisplay.from_predictions(
        y_true_mock, score, name=name, ax=ax
    )

ax.set_title('Detection Error Tradeoff (DET) Curve')
plt.grid(True, which='both', linestyle='--')
print("Curva DET gerada.")
```

Listing 7.2: Cálculo manual para curva DET customizada (escala linear/log)

```
def plot_manual_det(y_true, y_score, name):
    fpr, tpr, _ = roc_curve(y_true, y_score)
    fnr = 1 - tpr
    plt.plot(fpr, fnr, label=name)

plt.figure()
for name, score in scores.items():
    plot_manual_det(y_true_mock, score, name)
plt.yscale('log')
plt.xscale('log')
plt.xlabel('False Positive Rate (log)')
plt.ylabel('False Negative Rate (log)')
plt.legend()
plt.title('DET (Escala Log)')
plt.grid(True, which='both')
```

Capítulo 8

Distribuição de Scores

Ver a sobreposição entre as pontuações dos áudios reais (*bonafide*) e falsos (*spoof*) dá uma grande intuição sobre os erros.

Listing 8.1: Plotando histogramas de distribuição de scores

```
import seaborn as sns

plt.figure(figsize=(10, 5))
bonafide_scores = y_score_mock[y_true_mock == 0]
spoof_scores = y_score_mock[y_true_mock == 1]

sns.histplot(bonafide_scores, color='blue', label='Bonafide', kde=
    True, stat='density', alpha=0.5, bins=40)
sns.histplot(spoof_scores, color='red', label='Spoof', kde=True,
    stat='density', alpha=0.5, bins=40)

plt.axvline(x=eer_th, color='black', linestyle='--', label=f'EER
    Threshold ({eer_th:.2f})')
plt.xlabel('Score')
plt.ylabel('Densidade')
plt.title('Distribuicao de Scores')
plt.legend()
plt.tight_layout()
```

Listing 8.2: Verificando a taxa de sobreposição (Overlap)

```
# Um calculo heuristico de sobreposicao com base nos limites max/
    min
min_spoof = np.min(spoof_scores)
max_bona = np.max(bonafide_scores)

if max_bona > min_spoof:
    print(f"Area de confusao: [{min_spoof:.3f} , {max_bona:.3f}]")
else:
    print("Separacao linear perfeita!")
```

Listing 8.3: Violin Plots para comparação visual

```
df_scores = pd.DataFrame({
    'Score': np.concatenate([bonafide_scores, spoof_scores]),
```

```
    'Label': ['Bonafide']*len(bonafide_scores) + ['Spoof']*len(
        spoof_scores)
})

plt.figure(figsize=(8,6))
sns.violinplot(x='Label', y='Score', data=df_scores, palette='Set2'
    )
plt.title('Distribuicao (Violin Plot)')
```

Capítulo 9

EER por Sub-banda (Ablation Study)

O que acontece se removermos algumas sub-bandas? Treinamos novamente o modelo usando apenas sub-conjuntos das *features*.

Listing 9.1: Definindo *subsets* de *features* para ablação

```
# Suponha que queremos treinar sem as altas frequencias (sb6 e sb7)
features_no_high = [f for f in feature_names if 'sb6' not in f and
    'sb7' not in f]
idx_no_high = [feature_names.index(f) for f in features_no_high]

X_train_no_high = X_train_mock[:, idx_no_high]
X_test_no_high = X_test_mock[:, idx_no_high]
```

Listing 9.2: Treinando o modelo ablacionado

```
rf_ablated = RandomForestClassifier(n_estimators=50, random_state
    =42)
rf_ablated.fit(X_train_no_high, y_train_mock)

y_pred_no_high = rf_ablated.predict_proba(X_test_no_high)[:, 1]
eer_no_high, _ = compute_eer(y_test_mock, y_pred_no_high)

print(f"EER Original (Todas as features): {eer_val:.4f}")
print(f"EER sem Altas Frequencias: {eer_no_high:.4f}")
```

Listing 9.3: Ablação iterativa (leave-one-subband-out)

```
ablation_results = {}

for sb in range(num_subbands):
    features_to_keep = [f for f in feature_names if f'sb{sb}' not
        in f]
    idx_to_keep = [feature_names.index(f) for f in features_to_keep
        ]

    rf_lo = RandomForestClassifier(n_estimators=20, random_state
        =42)
    rf_lo.fit(X_train_mock[:, idx_to_keep], y_train_mock)

    preds = rf_lo.predict_proba(X_test_mock[:, idx_to_keep])[:, 1]
```

```
    eer_curr, _ = compute_eer(y_test_mock, preds)
    ablation_results[f"Sem sb{sb}"] = eer_curr

for k, v in ablation_results.items():
    print(f"{k}: EER = {v:.4f}")
```

Capítulo 10

Figuras Finais

A apresentação dos resultados num relatório ou artigo requer figuras limpas e claras.

Listing 10.1: Gráfico de barras de EER por Wavelet e Nível

```
# Agrupando os dados simulados
df_plot = df_results[df_results['classifier'] == 'RF']
pivot_df = df_plot.pivot(index='wavelet', columns='level', values='
    eer')

pivot_df.plot(kind='bar', figsize=(8, 6), colormap='viridis')
plt.ylabel('EER')
plt.xlabel('Wavelet M e')
plt.title('Desempenho (EER) por Wavelet e N vel de Decomposi  o'
    )
plt.legend(title='N vel WPT')
plt.tight_layout()
```

Listing 10.2: Heatmap de Importância (Sub-banda vs Estatística)

```
# Preparando matriz para heatmap
heatmap_data = np.zeros((num_subbands, len(stats_names)))

for i, sb in enumerate(range(num_subbands)):
    for j, st in enumerate(stats_names):
        feat = f"sb{sb}_{st}"
        idx = feature_names.index(feat)
        heatmap_data[i, j] = importances[idx]

plt.figure(figsize=(8, 6))
sns.heatmap(heatmap_data, annot=True, fmt=".3f", cmap='YlGnBu',
            xticklabels=stats_names, yticklabels=[f"sb{i}" for i in
                range(num_subbands)])
plt.title('Mapa de Calor da Import ncia das Features')
plt.xlabel('Estat stica')
plt.ylabel('Sub-banda')
plt.tight_layout()
```

Listing 10.3: Salvando figuras com qualidade de publicação (PDF/SVG)

```
# Exemplo gen rico de como salvar figuras
```

```
# plt.savefig('figura_det.pdf', format='pdf', dpi=300, bbox_inches='tight')
# plt.savefig('figura_det.svg', format='svg', bbox_inches='tight')
print("Lembre-se de salvar em PDF ou SVG para evitar perda de qualidade em texto.")
```

Capítulo 11

Brincadeiras (4 Experimentos)

Nesta seção, rodamos 4 miniexperimentos rápidos para sanar curiosidades.

Listing 11.1: Experimento 1: PCA antes do classificador melhora?

```
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline

pipe_pca = Pipeline([
    ('pca', PCA(n_components=0.95)), # Retem 95% da variancia
    ('rf', RandomForestClassifier(n_estimators=50, random_state=42)
       )
])

pipe_pca.fit(X_train_mock, y_train_mock)
pca_preds = pipe_pca.predict_proba(X_test_mock)[:, 1]
pca_eer, _ = compute_eer(y_test_mock, pca_preds)
print(f"EER com PCA: {pca_eer:.4f}")
```

Listing 11.2: Experimento 2: Qual a correlacao entre as *features*?

```
plt.figure(figsize=(12, 10))
corr_matrix = pd.DataFrame(X_train_mock, columns=feature_names).
   corr()
# Plota apenas uma mascara inferior para limpeza
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, cmap='coolwarm', vmin=-1, vmax
   =1)
plt.title("Matriz de Correla  o das Features")
```

Listing 11.3: Experimento 3: Testando um classificador levíssimo (Regressão Logística)

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

pipe_lr = Pipeline([
    ('scaler', StandardScaler()),
    ('lr', LogisticRegression(max_iter=1000))
])

pipe_lr.fit(X_train_mock, y_train_mock)
```

```
lr_preds = pipe_lr.predict_proba(X_test_mock)[:, 1]
lr_eer, _ = compute_eer(y_test_mock, lr_preds)
print(f"EER Regressao Logistica (Baseline baseline): {lr_eer:.4f}")
```

Listing 11.4: Experimento 4: Filtrando apenas Top 5 Features

```
top_5_features = importances_df['Feature'].head(5).values
top_5_idx = [feature_names.index(f) for f in top_5_features]

rf_top5 = RandomForestClassifier(n_estimators=50, random_state=42)
rf_top5.fit(X_train_mock[:, top_5_idx], y_train_mock)

y_pred_top5 = rf_top5.predict_proba(X_test_mock[:, top_5_idx])[:,
    1]
eer_top5, _ = compute_eer(y_test_mock, y_pred_top5)
print(f"EER usando APENAS as Top 5 Features: {eer_top5:.4f}")
```

Capítulo 12

Checklist

Todos os EERs e min-DCFs calculados para os dados de teste.

Tabela compilada e ordenada pelas melhores configurações (Tabela LaTeX gerada).

Feature Importance nativa do XGBoost/RF extraída e plotada.

Permutation Importance calculada no set de teste e analisada.

Identificação explícita da concentração de importância nas altas/baixas frequências.

Curvas ROC plotadas comparando ao menos os top 3 modelos.

Curvas DET geradas usando o padrão de publicação.

Histograma e *Violin plot* da distribuição de scores (*spoof* vs *bonafide*).

Estudo de ablação (leave-one-out) de sub-bandas rodado.

Todas as figuras salvas em formato vetorial (.pdf ou .svg) para o artigo final.

Capítulo 13

Desafio Final

Desafio

Baseado em toda a análise de contribuição: conseguimos criar um conjunto de *features* "Frankenstein"?
Ação: Pegue a melhor Wavelet/Nível e extraia suas descrições. Pegue as estatísticas que melhor performaram. Reduza o vetor apenas às 10 *features* cruciais selecionadas e crie um *ensemble* extremamente leve. Ele bate a baseline oficial do ASVspoof? Documente o EER deste "Frankenstein".

Listing 13.1: Código base para montar o modelo Frankenstein

```
# Baseado na analise, sabemos as top 10 features absolutas
frankenstein_idx = top_5_idx + [feature_names.index(f) for f in
    perm_df['Feature'].head(5)]
# Remover duplicatas
frankenstein_idx = list(set(frankenstein_idx))

print(f"Treinando Frankenstein com {len(frankenstein_idx)} features
    ")
rf_frank = RandomForestClassifier(n_estimators=100, max_depth=5,
    random_state=42)
rf_frank.fit(X_train_mock[:, frankenstein_idx], y_train_mock)

frank_preds = rf_frank.predict_proba(X_test_mock[:,
    frankenstein_idx])[:, 1]
frank_eer, _ = compute_eer(y_test_mock, frank_preds)
print(f"EER do Modelo Frankenstein: {frank_eer:.4f}")
```